



Stony Brook University

Bootstrap Resampling, Bagging & Random Forest

Weihaio Wang

**FAR
BEYOND**

Outlines

- **Bootstrap Resampling**
- Bagging
- Random Forest
 - Review of CART
 - Algorithm
 - Advantages and limitations
 - Examples & R codes

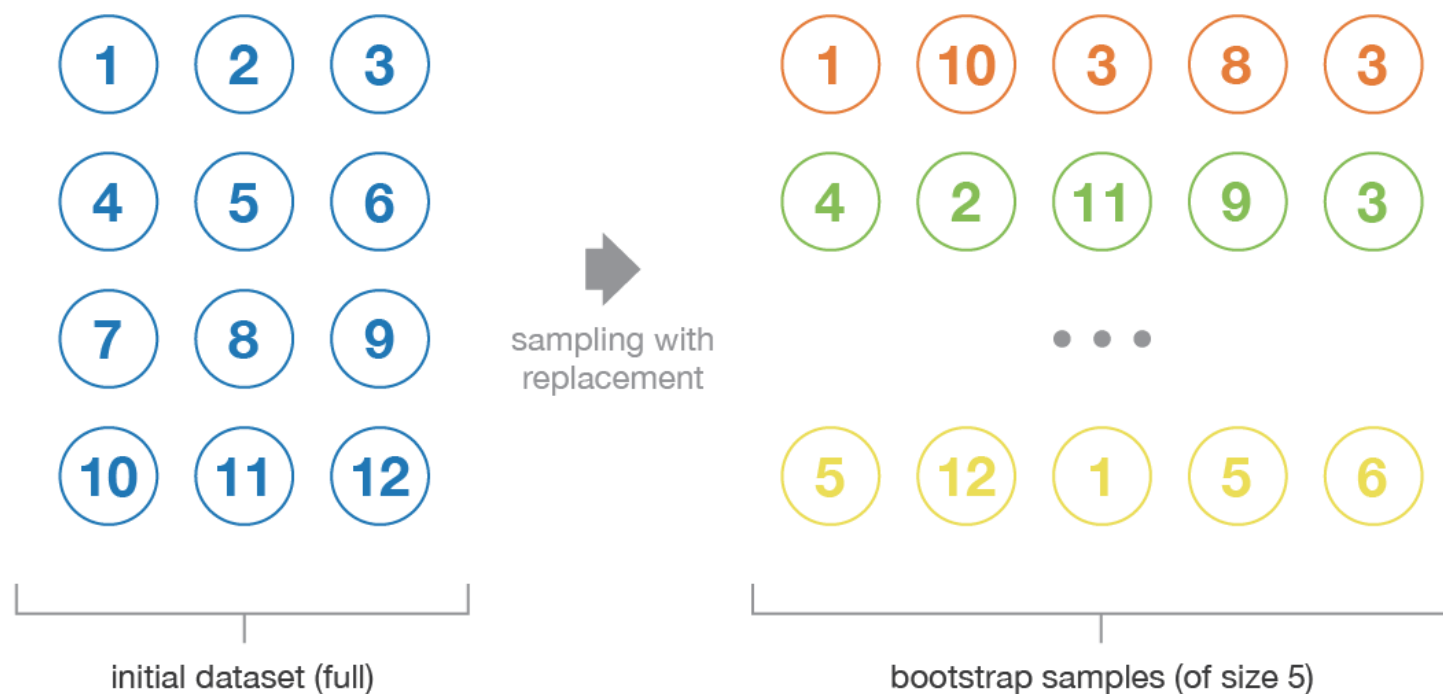
Bootstrap Resampling

- The bootstrap resampling ([https://en.wikipedia.org/wiki/Bootstrapping_\(statistics\)](https://en.wikipedia.org/wiki/Bootstrapping_(statistics))) was published by [Bradley Efron](#) in "Bootstrap methods: another look at the jackknife" (1979), inspired by earlier work on the [jackknife](#) resampling (https://en.wikipedia.org/wiki/Jackknife_resampling)
- In this method, we create new samples by randomly drawing samples from the original dataset **with replacement**. These new samples are usually of the same size as the original dataset, although their sizes can vary depending on the requirements of the analysis.
- Since samples are drawn with replacement, **some observations may appear multiple times** in a bootstrap sample, while others may be omitted.

Bootstrap Resampling

- The primary purpose of bootstrap resampling is to estimate the sampling distribution of a statistic or parameter **without making strong assumptions** about the underlying population distribution.
- It allows researchers to quantify the uncertainty associated with an estimator or test statistic by generating a large number of bootstrap replicates.

Bootstrap Resampling

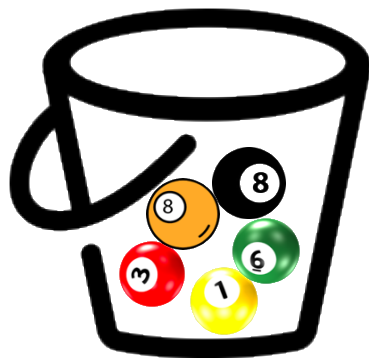


Note: In this example, we have a sample of size 12, and we resample for samples of size 5.

In applications, usually we will use the same sample size, 12 in this case.

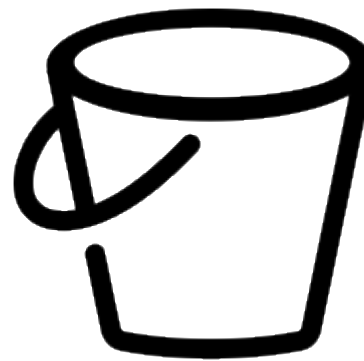
Bootstrap Resampling

Imagine we have 5 billiard balls in a bucket.

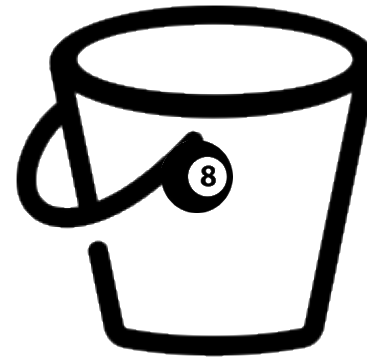
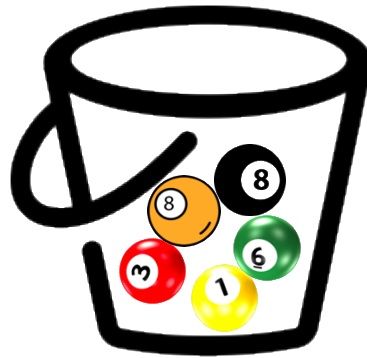


Bootstrap Resampling

We first pick randomly a ball and replicate it. This is called **sampling with replacement**. We move the replicated ball to another bucket.



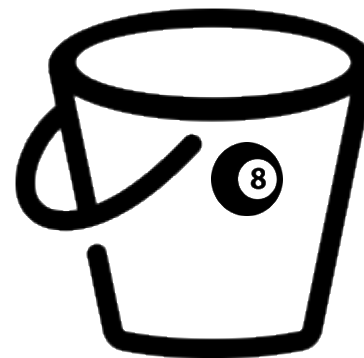
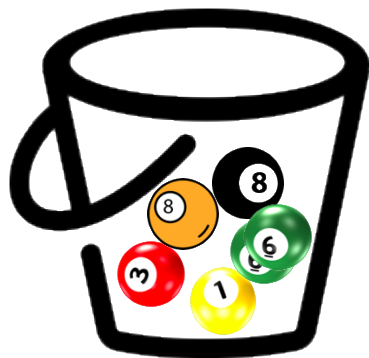
Bootstrap Resampling



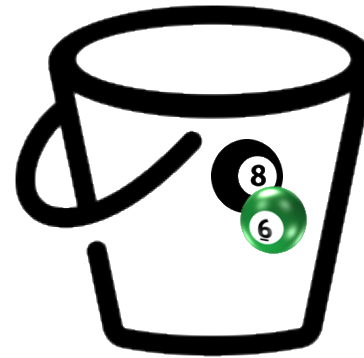
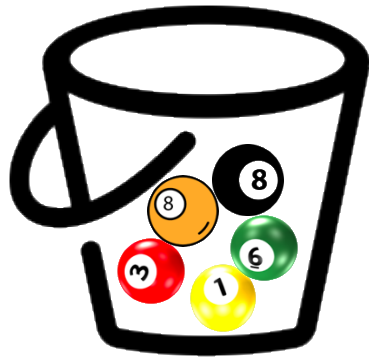
Bootstrap Resampling

We then randomly pick another ball and again we replicate it.

As before, we move the replicated ball to the other bucket.

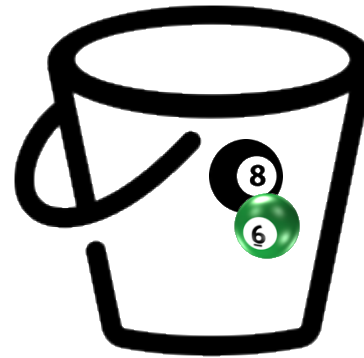


Bootstrap Resampling



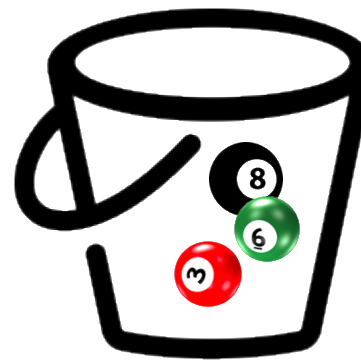
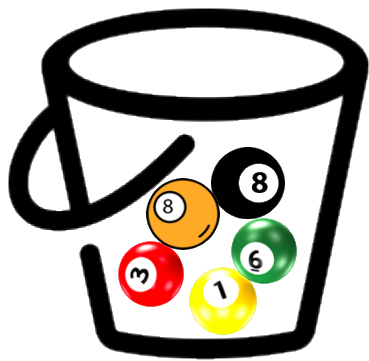
Bootstrap Resampling

We repeat this process.



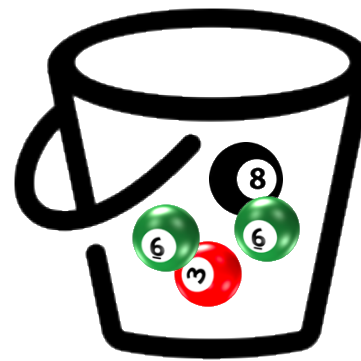
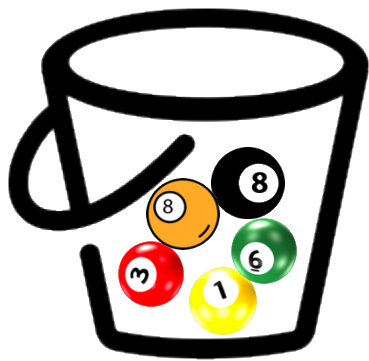
Bootstrap Resampling

Again



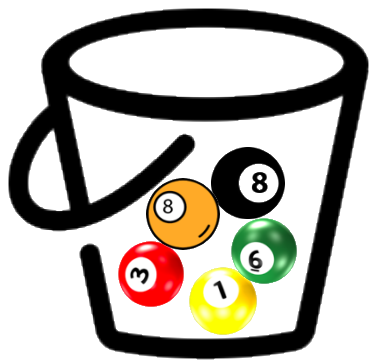
Bootstrap Resampling

And again



Bootstrap Resampling

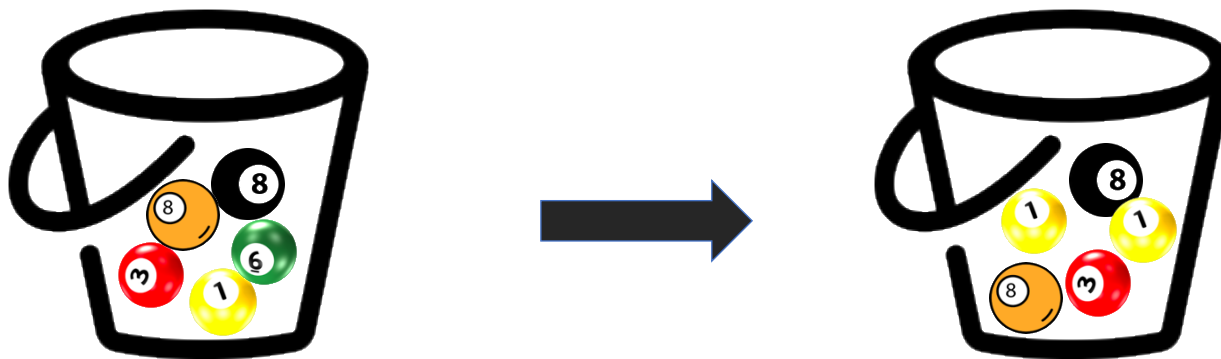
We continue until the “other” bucket has **the same number of balls** as the original one – because as mentioned before, **the most popular sample size for the Bootstrap resampling, is the same size as the original sample.**



This new bucket represents a new sample – note that some of the original sample points may not have been selected (**Ball 1 in this example**), while some others may appear more than once (**Ball 6 in this example**).

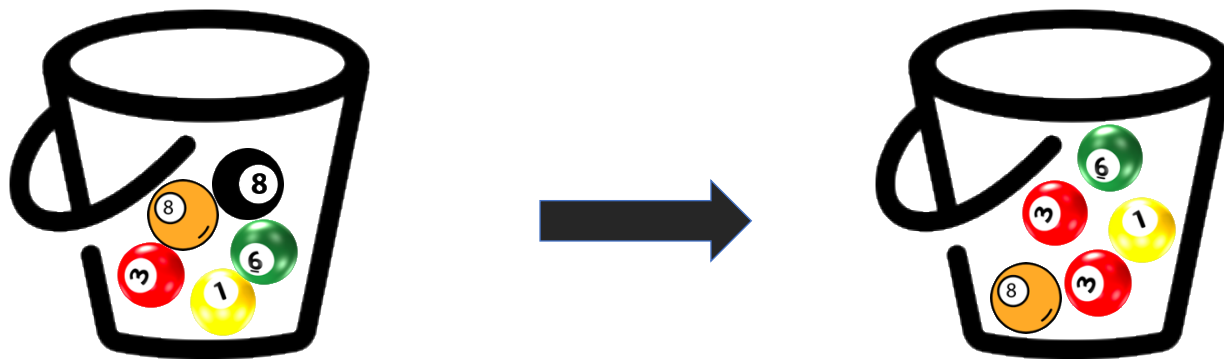
Bootstrap Resampling

We repeat the same process and acquire another sample.



Bootstrap Resampling

We repeat the same process and acquire another sample.



Outlines

- Bootstrap Resampling
- **Bagging**
- Random Forest
 - Review of CART
 - Algorithm
 - Advantages and limitations
 - Examples & R codes

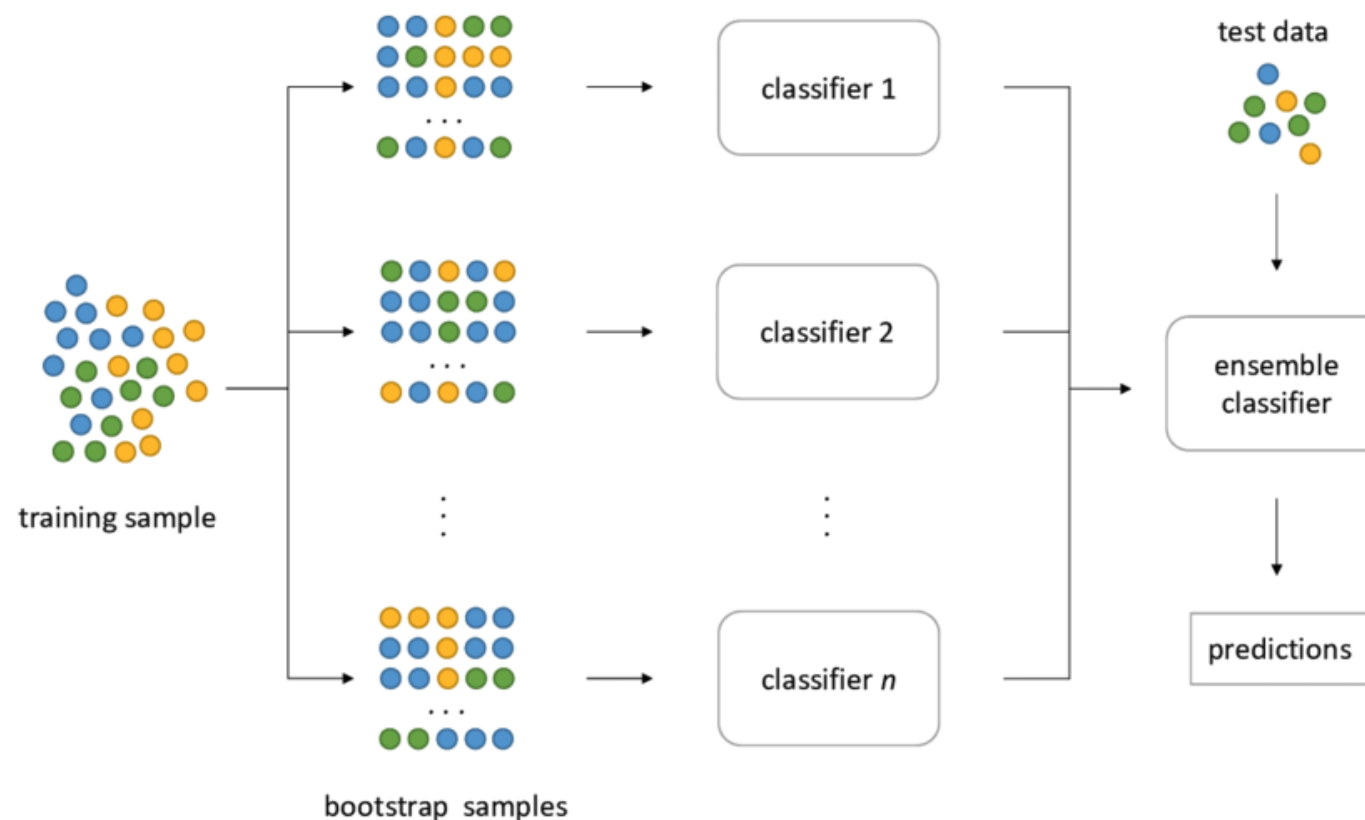
Bagging

Bootstrap aggregating, also called **bagging** (from **bootstrap aggregating**), is a popular **ensemble learning** technique that combines the predictions of multiple base learners to improve the overall performance and **robustness** of the model.

Bagging involves training multiple instances of the same base learning algorithm on different subsets of the training data, usually through bootstrap resampling, and then **averaging** or **voting** the predictions of the individual models to make the final prediction.

Bagging

- Bagging works by creating **multiple bootstrap samples** from the original training dataset.
- Each bootstrap sample is used to train a base learner, such as a decision tree, neural network, or regression model.
- The final prediction is obtained by **averaging** (for regression) or **voting** (for classification) the predictions of the individual models.



Bagging

- Bagging helps **reduce variance** and **improve the stability and generalization** of the model by averaging out the predictions of multiple models.
- It is particularly effective when the base learners are high-variance models (models that tend to overfit the training data).

Ensemble Learning

- **Ensemble learning** involves training multiple base learners independently and then combining their predictions to make a final prediction.
- The **diversity** among the base learners is crucial for the success of ensemble methods, as it allows different models to capture different aspects of the data or make different errors.
- Bagging (Bootstrap Aggregating): Combines predictions from multiple models trained on bootstrap samples of the data.
- Random Forest: A specific ensemble method that uses bagging with decision trees as the base learners.

Outlines

- Bootstrap Resampling
- Bagging
- **Random Forest**
 - Review of CART
 - Algorithm
 - Advantages and limitations
 - Examples & R codes

Random Forest

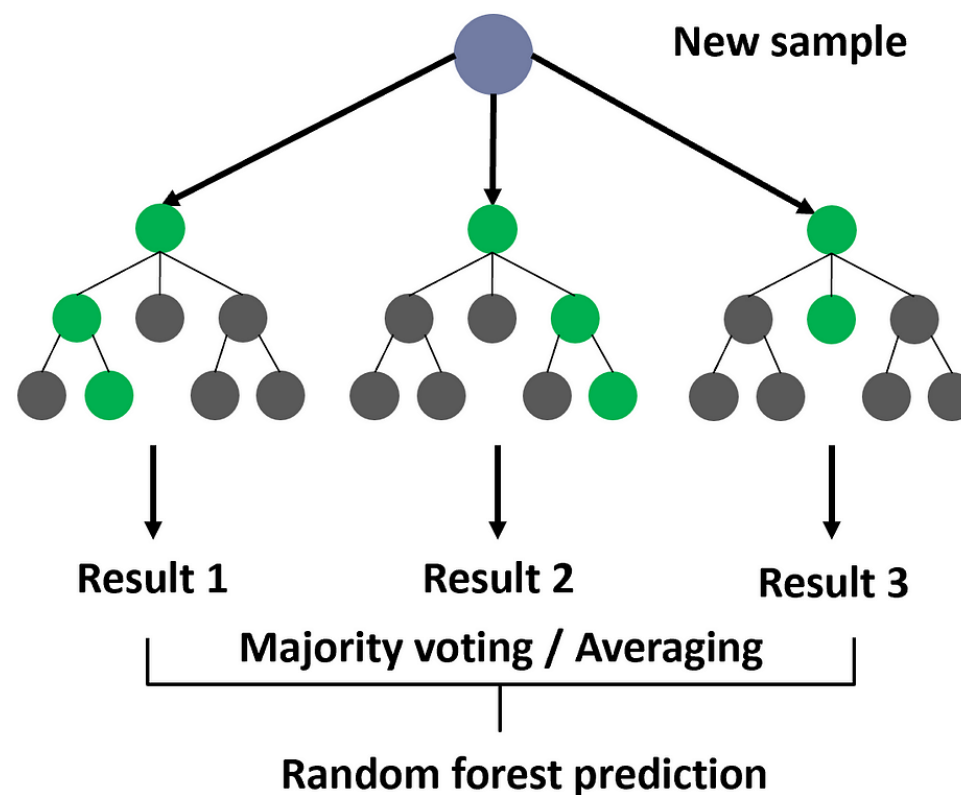
Random forest (or **random forests**) is an ensemble classifier that consists of many decision trees and outputs the class that is the mode of the class's output by individual trees.

The term came from **random decision forests** that was first proposed by Tin Kam Ho of Bell Labs in 1995.

The method combines Breiman's "**bagging**" idea and the **random selection of features**. (2001)

Random Forest builds upon the idea of **bagging**, where multiple decision trees are trained on different subsets of the data to reduce overfitting and improve predictive performance.

In the case of Random Forest, these decision trees are specifically constructed using the **CART** (Classification and Regression Trees) algorithm.

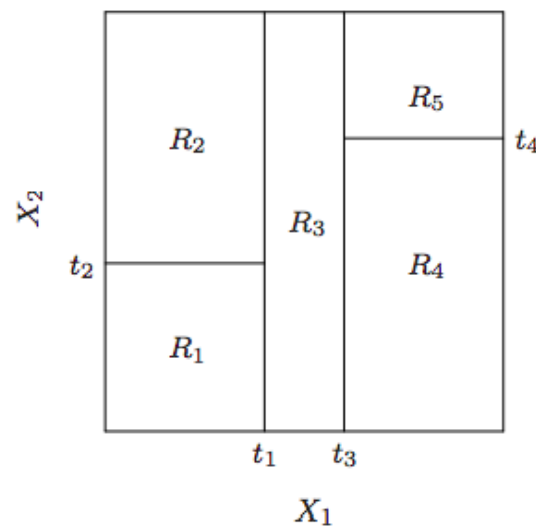
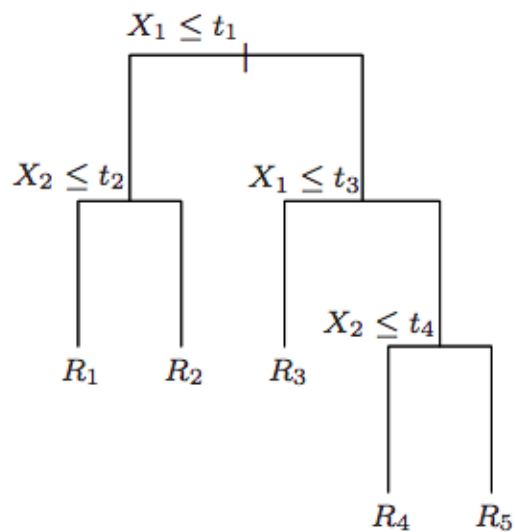


Outlines

- Bootstrap Resampling
- Bagging
- **Random Forest**
 - **Review of CART**
 - Algorithm
 - Advantages and limitations
 - Examples & R codes

Review of Decision Trees (CART)

CART is the decision tree of choice for constructing a forest. It is a greedy, top-down, binary recursive algorithm that divides the feature space into disjoint rectangular regions.



How to determine the best split?

In CART (Classification and Regression Trees), **impurity measures** are used to evaluate the homogeneity of a node in a decision tree. The impurity measures help determine the best split at each node by selecting the feature and the split value that result in the highest reduction in impurity. The two commonly used impurity measures in CART are Gini and Entropy:

Gini Impurity:

- Gini impurity measures the probability of incorrectly classifying a randomly chosen element if it was randomly labeled according to the distribution of labels in the node.
- It is defined as the sum of the squared probabilities of each class being chosen, subtracted from 1.
- Mathematically, for a node with K classes, the Gini impurity I_G is calculated as:

$$I_G = 1 - \sum_{i=1}^K p(i)^2$$

- A lower Gini impurity indicates a more homogeneous node.

How to determine the best split?

Entropy:

- Entropy measures the level of disorder or randomness in the distribution of class labels in a node.
- It is defined as the sum of the probabilities of each class label multiplied by the logarithm of the probability of that class label.
- Mathematically, for a node with K classes, the entropy I_H is calculated as:

$$I_H = - \sum_{i=1}^K p(i) \log_2(p(i))$$

- A lower entropy indicates a more homogeneous node.

Both Gini impurity and entropy are commonly used in CART algorithms to determine the optimal split. While **Gini impurity** tends to be **faster** to compute, **entropy** can be more **sensitive to changes** in class probabilities. In practice, the choice between the two impurity measures may depend on the specific dataset and problem domain.

Advantages of CART

- No formal distribution assumptions
- Can fit highly non-linear interactions
- Unified tool for regression and classification
- Handles categorical and continuous variables
- Not impacted by outliers in input variables
- Easily Interpretable

Limitations of CART

- **High Variance and Overfitting:**

CART decision trees can be prone to overfitting, especially when the trees are deep or when the dataset is noisy. This can lead to poor generalization performance on unseen data.

- **Instability:**

Small changes in the training data can lead to significantly different trees being generated. This instability can make the model sensitive to variations in the dataset and may require techniques such as ensemble methods to mitigate.

Random Forest to the Rescue!

- **High Variance and Overfitting:**

Random Forest mitigates overfitting by constructing multiple decision trees on **bootstrapped** samples of the dataset and averaging their predictions!

→ No Pruning of the Trees!

- **Instability:**

The ensemble nature of Random Forest makes it less sensitive to small changes in the training data, resulting in more robust and stable predictions.

→ Increased Model Accuracy!

What makes it random?

1. Bagging (Bootstrap Aggregating): Bagging is a technique where multiple subsets of the original dataset are created by **sampling with replacement** (bootstrap sampling). Each subset is used to train a separate decision tree. This process introduces randomness into the training process because each tree is trained on a slightly different dataset.

2. Random Feature Selection: In addition to bagging, Random Forest further randomizes the construction of individual trees by considering **only a random subset of features at each split** in the decision tree. This means that at each node of the tree, the algorithm selects only a subset of features rather than considering all features.

OOB

- If the training sample size N is large, then the percent of observations in the original training sample not selected in each tree (resample of size N) equals to:

$$100\left(1 - \frac{1}{N}\right)^N \approx 100\left(\frac{1}{e}\right) \approx 37\%$$

- These excluded cases are called the “out of bag (OOB) sample” and they come in handy in estimating the error rate for the training data
- In our class, we split the data into training and testing and use the OOB to estimate the error rate of the training sample (-- although there are people who use the entire data to perform the RF and use the OOB error rate to estimate the overall error rate.)

How to test the model performance?

- Roughly 1/3 of the (training) sample cases are left out of each tree (called **out of bag sample** or **OOB sample**). This sample serves as an **internal validation** method, estimating the error rate of the Random Forest without the need for traditional cross-validation or additional test data.
- For every sample case not used in constructing the k^{th} tree, it is run down that tree. This process is repeated for all trees. Let's denote j as the class that receives the highest number of votes for case n . The **out-of-bag error rate** is then calculated as the proportion of times j does not match the true response value of case n , averaged across all cases.
- This internal validation method is unbiased and equally effective as using separate testing data. **However, in our class, we still employ a distinct testing dataset to further validate our Random Forest model.**

Outlines

- Bootstrap Resampling
- Bagging
- **Random Forest**
 - Review of CART
 - **Algorithm**
 - Advantages and limitations
 - Examples & R codes

The Random Forest Algorithm

1.Input: Training dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, where x_i represents the predictors and y_i represents the corresponding response.

2.Random Feature Selection: For each tree in the forest:

- **Randomly select a subset of features** of size m from the total M features available.
- m is typically much smaller than M , often chosen as the square root of M or a constant value determined empirically.

3.Bootstrap Sampling: For each tree in the forest:

- Draw a **bootstrap sample** D_i of size n from the training dataset D **with replacement**. This sample will contain n instances randomly selected from D , possibly with duplicates and omissions.

The Random Forest Algorithm

4. Tree Construction: For each tree in the forest:

- Grow a **decision tree** using the CART (Classification and Regression Trees) algorithm on the bootstrap sample D_i .
- At each node of the tree:
 - Randomly select m features from the subset chosen earlier.
 - Choose the best feature and split point based on a chosen criterion (e.g., Gini impurity for classification, mean squared error for regression).

5. Prediction Aggregation:

- For classification tasks:
 - Use a **majority voting** scheme to determine the final class label. Each tree "votes" for a class label, and the class with the most votes becomes the final prediction.
- For regression tasks:
 - Calculate the **average** of the predicted values from all trees to obtain the final prediction.

The RF Algorithm

(Diagram customized by Dr.Hu & Prof.Zhu)

All Training Cases
Sample IDs
{1,2,3,4,5,6,7,8,9,10}
 $N = 10$

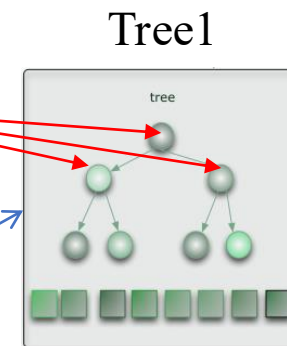
Bootstrap sample 1: {1,8,2,10,3,1,2,7,10,8}

$N = 10$

OOB (Out of Bag) samples: {4,5,6,9}

Select a subset
of m variables

Build



Drop down the tree 1

⋮

⋮

OOB
error

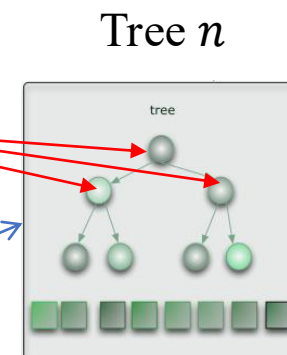
Bootstrap sample n : {2,7,4,9,7,5,4,2,5,8}

$N = 10$

OOB (Out of Bag) samples: {1,3,6,10}

Select a subset
of m variables

Build



Drop down the tree n

Tuning Parameters

1. Number of Trees (ntree):

- The number of trees in the Random Forest model.
- Increasing the number of trees can improve the **robustness** of the model and reduce the risk of **overfitting**.
- However, training time and computational resources also increase with the number of trees.

2. Number of Variables Randomly Sampled (mtry):

- The number of predictor variables randomly selected as candidates at each split of a decision tree.
- Controlling the randomness in feature selection helps reduce correlation among trees and improves the diversity of the ensemble.
- A common rule of thumb is to set *mtry* to the square root of the total number of predictors.

Tuning Parameters

3. Minimum Node Size (nodesize):

- The minimum number of observations required to split a node further.
- Increasing the minimum node size can prevent the model from **overfitting to noisy data** by limiting the complexity of individual trees.
- A larger node size results in simpler trees with fewer splits, which may improve interpretability but could sacrifice predictive performance.

4. Maximum Depth of Trees (maxdepth):

- The maximum depth allowed for each decision tree in the Random Forest.
- Limiting the tree depth helps **prevent overfitting** by controlling the complexity of individual trees.
- However, setting a maximum depth that is too shallow may lead to underfitting, where the model fails to capture complex relationships in the data.

Tuning Parameters

5. Sample Size (sampsiz):

- The size of the bootstrap sample used to train each tree.
- Specifying a smaller sample size can reduce training time and computational resources, especially for large datasets.
- However, excessively small sample sizes may lead to biased estimates and increased variability in the model.

6. Other Parameters:

- Splitting criterion (e.g., Gini impurity or entropy).
- Class weights (to address class imbalance).
- Variable importance calculation method.
- Parallelization options (for training on multi-core processors).

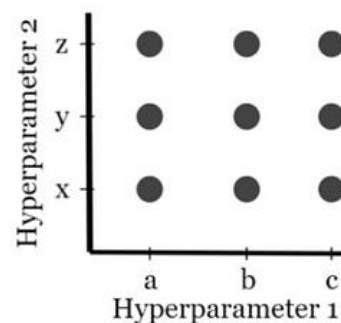
Tuning Parameters

7. Parameter Tuning Methods:

1. **Grid search:** Evaluates the model performance across a predefined grid of parameter values and selects the combination that yields the best performance.
2. Random search: Randomly samples parameter values from specified distributions and evaluates the model performance for each sampled combination.
3. Bayesian optimization: Employs probabilistic models to guide the search for optimal parameter values based on past evaluations.

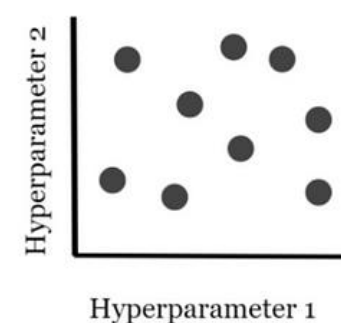
Grid Search

Pseudocode
Hyperparameter_One = [a, b, c]
Hyperparameter_Two = [x, y, z]



Random Search

Pseudocode
Hyperparameter_One = random.num(range)
Hyperparameter_Two = random.num(range)



Source: Grid search CV Vs
Randomize search CV, Kumar, R.

Outlines

- Bootstrap Resampling
- Bagging
- **Random Forest**
 - Review of CART
 - Algorithm
 - **Advantages and limitations**
 - Examples & R codes

Advantages of RF

- High accuracy
- Robustness to overfitting
- Handles large datasets
- Provides feature importance
- Reduces Variance
- Non-parametric and versatile
- Handles missing values
- Outlier robustness

Limitations of RF

- **Visualization** is difficult.
- **Interpretability** is not as straightforward as that of an individual tree, where the influence of a predictor variable directly corresponds to its position in the tree
- **Limited Performance on Imbalanced Data.** Random Forest may struggle to accurately predict minority classes in imbalanced datasets, where one class significantly outweighs the others. The majority class tends to dominate the decision-making process, leading to biased predictions.

Limitations of RF

- Visualization ← feature importance plots & tree visualization
- Interpretability ← feature importance ranking
- Imbalanced Data ← class weighting & sampling techniques

Variable/feature Importance

Random Forest employs two primary methods for calculating variable importance:

- **Mean Decrease in Impurity** (MDI): Also known as Gini importance, MDI measures the decrease in node impurity (e.g., Gini impurity or entropy) achieved by splitting on a particular feature across all trees in the forest. Features with higher MDI values are considered more important.
- **Mean Decrease in Accuracy** (MDA): MDA evaluates the decrease in prediction accuracy achieved by permuting (randomly shuffling) the values of a particular feature while keeping other variables unchanged. Features with larger decreases in accuracy after permutation are deemed more important.

Outlines

- Bootstrap Resampling
- Bagging
- **Random Forest**
 - Review of CART
 - Algorithm
 - Advantages and limitations
 - **Examples & R codes**

Random Forest Example

1.randomForest package: This is the primary package for implementing Random Forest in R. It provides the main function `randomForest()` to build Random Forest models.

- **randomForest():** This function builds a Random Forest model. It takes input variables and a response variable as arguments, along with optional parameters like the number of trees (`ntree`), the number of variables randomly sampled as candidates at each split (`mtry`), and others.

2.caret package: The caret package is a comprehensive toolkit for machine learning tasks in R. While it doesn't directly build Random Forest models, it provides convenient functions for pre-processing, model training, and evaluation.

- **train():** This function from the caret package is used to train various machine learning models, including Random Forest. It takes as input the formula specifying the model, the training dataset, and optional parameters.

Random Forest Example

```
library(randomForest)
```

```
rf <- randomForest(y ~ ., data = train.data, ntree = 100, mtry = 2)
```

- **ntree**: The number of trees to grow in the forest.
- **mtry**: The number of variables randomly sampled as candidates at each split.

```
library(caret)
```

```
rf <- train(y ~ ., data = train.data, method = "rf", trControl = trainControl(method = "cv", number = 10),  
tuneGrid = expand.grid(ntree = c(50, 100), mtry = c(2, 3)))
```

- **trControl**: Controls the computational settings for the training process. It includes parameters like cross-validation settings, resampling method, etc.
- **tuneGrid**: Specifies a grid of parameters over which to optimize the model. For Random Forest, you can specify parameters like ntree and mtry here.

Random Forest Example

The dataset used in this analysis is sourced from the Kaggle competition platform (<https://www.kaggle.com/c/titanic>). It comprises information on 891 passengers and encompasses 12 variables:

PassengerId: Unique identifier assigned to each passenger, ranging from 1 to 891.

Survived: Binary variable denoting whether the passenger survived (1) or not (0); this serves as the **response variable**.

Pclass: Class of the passenger (1 = 1st class; 2 = 2nd class; 3 = 3rd class).

Name: Comprehensive field containing the passenger's title and family names, providing valuable information.

Sex: Gender of the passenger, classified as male or female.

Age: Age of the passenger; noteworthy to mention that a substantial number of values are missing.

SibSp: Count of siblings/spouses accompanying the passenger onboard.

Parch: Count of parents/children accompanying the passenger onboard.

Ticket: Ticket number issued to the passenger.

Fare: Fare paid by the passenger for the journey, denoted in British Pound (£).

Cabin: Cabin number assigned to the passenger.

Embarked: Port from which the passenger embarked (C = Cherbourg; Q = Queenstown; S = Southampton).

Random Forest Example

```
library(caret)
library(randomForest)
```

```
data <- read.csv('Titanic.csv') # read the dataset from a CSV file
data <- subset(data, select = -c(PassengerId, Name, Ticket, Cabin))
data$Survived <- as.factor(data$Survived)
```

```
# classification
```

```
set.seed(123) # produce the same results each time
```

```
split <- data$Survived %>% createDataPartition(p = 0.75, list = FALSE)
```

```
train <- data[split,]
```

```
test <- data[-split,]
```

```
# train the RF
```

```
model <- train(
```

```
  Survived ~., data = train, method = "rf",
```

```
  trControl = trainControl("cv", number = 10), # 10-fold CV
```

```
  importance = TRUE) # feature importance should be calculated
```

```
'data.frame':  891 obs. of  12 variables:
 $ PassengerId: int  1 2 3 4 5 6 7 8 9 10 ...
 $ Survived   : int  0 1 1 1 0 0 0 0 1 1 ...
 $ Pclass     : int  3 1 3 1 3 3 1 3 3 2 ...
 $ Name       : chr  "Braund, Mr. Owen Harris" "Cumings, Mrs. John Bradley"
Miss. Laina" "Futrelle, Mrs. Jacques Heath (Lily May Peel)" ...
 $ Sex        : chr  "male" "female" "female" "female" ...
 $ Age        : num  22 38 26 35 35 NA 54 2 27 14 ...
 $ SibSp      : int  1 1 0 1 0 0 0 3 0 1 ...
 $ Parch      : int  0 0 0 0 0 0 0 1 2 0 ...
 $ Ticket     : chr  "A/5 21171" "PC 17599" "STON/O2. 3101282" "113803" ...
 $ Fare       : num  7.25 71.28 7.92 53.1 8.05 ...
 $ Cabin      : chr  "" "C85" "" "C123" ...
 $ Embarked   : chr  "S" "C" "S" "S" ...
```

Random Forest Example

```
model$bestTune # the optimal hyperparameters selected
model$finalModel
```

```
pred <- model %>% predict(test)
confusionMatrix(factor(pred), factor(test$Survived))
```

Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	90	19
1	16	53

Accuracy : 0.8034
 95% CI : (0.7373, 0.8591)
 No Information Rate : 0.5955
 P-Value [Acc > NIR] : 2.711e-09

Kappa : 0.5891

Mcnemar's Test P-Value : 0.7353

Sensitivity : 0.8491
 Specificity : 0.7361
 Pos Pred Value : 0.8257
 Neg Pred Value : 0.7681
 Prevalence : 0.5955
 Detection Rate : 0.5056
 Detection Prevalence : 0.6124
 Balanced Accuracy : 0.7926

'Positive' Class : 0

mtry
<dbl>
2

number of variables randomly
sampled at each split

Call:

```
randomForest(x = x, y = y, mtry = param$mtry, importance = TRUE)
Type of random forest: classification
```

Number of trees: 500

No. of variables tried at each split: 2

OOB estimate of error rate: 21.27%

Confusion matrix:

	0	1	class.error
0	284	34	0.1069182
1	80	138	0.3669725

The **confusion matrix** is a table that summarizes the performance of a classification model by counting the number of true positives, true negatives, false positives, and false negatives. It provides insights into the model's accuracy, precision, recall, and other performance metrics.

Random Forest Example

```
varImpPlot(model$finalModel, type = 1) # Plot Mean Decrease Accuracy
```

```
varImpPlot(model$finalModel, type = 2) # Plot Mean Decrease Gini
```

